

Arkanoid in HAN-Yaeger

Functioneel-/ en Technisch ontwerp



Auteurs:

Robin Bravenboer (1580537)

Stephan Drost (2150188)

Docent: Peter Cornelissen

Cursus: Object Georiënteerd Programmeren (OGP)

Datum: 10-06-2026

Inhoudsopgave

1.0 Inleiding	2
Technische informatie.....	2
Opzet van dit document.....	2
2.0 Spelbeschrijving.....	3
2.1 Inleiding	3
2.2 Doel van het spel.....	3
2.3 Speldesign en levels	3
2.4 Objecten	4
3.0 Functionele eisen	5
4.0 Wireframes	7
4.1 Startscherm.....	7
4.2 Level Selecteren	8
4.3 Spelscherm.....	9
5.0 Technisch ontwerp.....	11
5.1 Inleiding	11
5.2 Prioriterisering	11
5.3 Klassenbeschrijving	13
6.0 Conclusie	17
7.0 Bijlage	18
7.1 Klassendiagram	18

1.0 Inleiding

Voor het vak OGP hebben wij de opdracht gekregen om een spel op te stellen waarbij als basis de Yaeger Game Engine gebruikt moet worden. Door gebruik te maken van object georiënteerde ontwikkeltechnieken gaan wij het spel Arkanoid realiseren.

Technische informatie

De programmeertaal die wij hanteren is Java en als game engine gebruiken wij HAN-Yaeger. Hierin is al een hoop logica en klassen voorgedefinieerd waar wij op kunnen baseren.

Daarnaast is er gekozen voor Arkanoid omdat dit spel zich goed leent voor object georiënteerd programmeren. Het spel bestaat uit verschillende losse onderdelen zoals de bal, paddle, blokken en levels, die allemaal eigen eigenschappen en gedrag bevatten. Hierdoor kunnen concepten zoals overerving, encapsulatie en samenwerking tussen objecten goed toegepast worden binnen het project.

Opzet van dit document

Dit document dient als leidraad voor het maken van onze versie van Arkanoid. Hierin staat beschreven hoe het spel speelt, wat de eisen zijn en hoe die te realiseren vallen. Door middel van een functioneel ontwerp en technisch ontwerp moet het mogelijk zijn om ons spel na te maken.

In hoofdstuk 2, 3 en 4 wordt toegelicht op functionele basis wat het idee is van het spel en in hoofdstuk 5 is het technisch ontwerp te vinden die een klassendiagram, plan van aanpak en functionele eisen bevat. Alles om onze versie van Arkanoid te maken.

Het doel van dit project is niet alleen het ontwikkelen van een werkend spel, maar ook het opdoen van ervaring met het ontwerpen van software volgens object georiënteerde principes. Tijdens het ontwikkelen wordt aandacht besteed aan structuur, herbruikbaarheid van code en onderhoudbaarheid van het project.

Tot slot is er nog een conclusie waarin wij aangeven tegen welke uitdagingen wij zijn gelopen en of wij bepaalde functioneleiten anders hebben ingevuld dan van tevoren besloten.

2.0 Spelbeschrijving

2.1 Inleiding

In 1986 kwam het originele spel Arkanoid uit gemaakt door Taito. Geïnspireerd door het toen mega Populaire spel Breakout leent het ook veel van het soortgelijke arcade spel.

Tot de huidige dag is het nog steeds een schitterend spel in retro spel verzamelingen. Hierbij wilden wij onze draai aan het spel geven als viering van het 40-jarig bestaan van Arkanoid.

Hoewel het originele spel relatief eenvoudig oogt, staat Arkanoid bekend om de combinatie van reflexen, timing en strategie. Door de jaren heen zijn meerdere varianten van het spel uitgebracht, waarbij nieuwe levels, power-ups en spelmechanieken zijn toegevoegd.

2.2 Doel van het spel

Het doel van het spel is het oplossen van puzzels op een andere manier dat men normaal gewend is. Hierbij is de puzzel het vernietigen van alle blokjes en het level te halen zonder het balletje te laten verdwijnen in de onderkant van het scherm. Het balletje in het spel houden wordt gedaan door een platform (ookwel “Vaus” genoemd in het originele spel).

Naast het behalen van levels speelt ook het verzamelen van punten een belangrijke rol binnen het spel. Punten worden verdiend door blokjes te vernietigen. Hierdoor wordt de speler gestimuleerd om levels zo efficiënt mogelijk te voltooien.

2.3 Speldesign en levels

In arkanoid bevindt zich de speler onderaan het scherm als Vaus en moet een balletje lanceren en terugkaatsen richting blokken die bovenaan het scherm staan te vernietigen. Door de onvoorspelbaarheid in het stuiteren van een balletje op blokjes kan het lastig zijn om het balletje niet langs de speler te laten passeren en zorgt zo voor een uitdagend spelverloop. Afhankelijk van hoe de blokjes aanwezig zijn op het speelveld kan het ook lastig zijn om het balletje gericht terug te kaatsen naar de plek waar nog blokjes zijn.

Naarmate de speler verder komt in het spel zullen levels moeilijker worden. Dit kan worden bereikt door complexere patronen van blokjes, sterkere blokken of een hogere snelheid van het balletje. Hierdoor blijft het spel uitdagend en wordt de moeilijkheidsgraad geleidelijk opgebouwd.

2.4 Objecten

Het spel bevat de volgende objecten:

- Platform ("Vaus")
- Balletje
- Blokjes
- Power-ups

Het platform ("Vaus")

Dit is de speler en het platform is enkel te bewegen in horizontale richtingen en beweegt in vergelijking met het balletje relatief traag om de moeilijkheid van het spel te behouden.

Balletje

Het balletje is altijd in beweging en stuitert van alle oppervlaktes in het spel af. Het balletje mag nooit onder het platform vallen want dan is het spel voorbij op basis van het aantal beschikbare levens.

Blokjes

Blokjes vormen de basis van elk level, er zijn meerder soorten blokjes, afhankelijk van het blokje zijn ze onverwoestbaar, moeten ze vaker dan 1x geraakt worden of kunnen ze een power-bevatten. Natuurlijk is er ook het normale blokje wat bij enkele aanraking kapotgaat en punten geeft aan de speler.

Power-ups

Power-ups geven tijdelijke voordelen aan de speler tijdens het spelen. Voorbeelden hiervan zijn een groter platform, meerdere balletjes tegelijk of een langzamer bewegend balletje. Power-ups verschijnen wanneer specifieke blokjes vernietigd worden.

Score systeem

Het score systeem houdt bij hoeveel punten de speler heeft verdiend tijdens het spelen. Door blokjes te vernietigen en power-ups te verzamelen loopt de score op. Aan het einde van het spel kan de score gebruikt worden om prestaties van spelers met elkaar te vergelijken.

3.0 Functionele eisen

In dit hoofdstuk worden de functionele eisen van het spel beschreven. Deze eisen geven aan welke functionaliteiten het spel minimaal moet bevatten om correct te kunnen functioneren en welke extra functionaliteiten bijdragen aan de speelervaring van de speler.

Om overzicht te houden in de prioriteiten van de functionaliteiten wordt gebruik gemaakt van de MoSCoW-methode. Hierbij worden de eisen verdeeld in Must have, Should have, Could have en Won't have. Op deze manier wordt duidelijk welke onderdelen essentieel zijn voor een werkende versie van het spel en welke onderdelen als uitbreiding of extra functionaliteit worden beschouwd.

Nr.	Eis	Prioriteit	Reden
1	Het spel moet afhankelijk van het level de juiste hoeveelheid blokken kunnen generen die wanneer ze in aanraking komen met het balletje een actie ondernemen	M	Basisfunctionaliteit van het spel
2	Het spel moet te besturen zijn met de pijltjestoetsen van een toetsenbord	M	Basisfunctionaliteit van het spel
3	Het spel moet een balletje intekenen die in rechte lijnen beweegt en bij botsing met een object in een gelijke hoek verder beweegt	M	Basisfunctionaliteit van het spel
4	Het spel moet een platform intekenen die te besturen is door de speler	M	Basisfunctionaliteit van het spel
5	Het spel moet afgekaderd zijn met onverwoestbare muren waar het balletje en het platform beiden niet doorheen kunnen	M	Basisfunctionaliteit van het spel
6	Het spel moet bijhouden of het balletje het speelveld verlaat	M	Basisfunctionaliteit van het spel
7	Het spel moet bijhouden of er balletjes op het speelveld bestaan	M	Basisfunctionaliteit van het spel
8	Het spel moet wanneer er geen balletjes meer op het speelveld aanwezig zijn een game-over scherm tonen waarop de behaalde score te zien is	M	Basisfunctionaliteit van het spel
9	Het spel moet een startscherm hebben	M	Basisfunctionaliteit van het spel
10	Het balletje moet ook vanaf de zijkant van het platform kunnen stuiteren om zo de bal weer terug te sturen waar die vandaan kwam	M	Basisfunctionaliteit van het spel
11	Bij spelstart moet het balletje aan het platform plakken om zo een zuivere start van het spel te garanderen	S	Kenmerk van het spel

12	Het spel moet een live score bijhouden	S	Kenmerk van spellen
13	Het spel heeft meerdere levels	S	Kenmerk van het spel
14	Bij het behalen van 1 level wordt een scorescherf getoond en heeft de speler de optie om door te spelen naar het volgende level of om te stoppen.	S	Enkel mogelijk als er meerdere levels zijn
15	De te verwoesten blokjes moeten willekeurig wel of niet een power-up activeren voor de speler	S	Kenmerk van het spel
16	Het spel moet een startscherm hebben waarin de highscores te zien zijn	C	Leuk om te hebben maar niet nodig
17	Een van de power-ups geeft de speler een laser waarmee de speler een gericht blok kan kapotmaken	S	Kenmerk van het spel
18	Het spel moet buiten een raster aan blokken ook andere designs voor levels moeten kunnen genereren om moeilijkere levels te creëren	S	Kenmerk van het spel
19	Het spel moet willekeurig blokken genereren die een power-up bezitten of niet verwoestbaar zijn.	S	Kenmerk van het spel
20	Het platform kan zich veranderen van formaat afhankelijk welke moeilijkheidsgraad gekozen is	C	Leuk om te hebben maar niet nodig
21	Het spel moet meerdere moeilijkheidsniveaus hebben	C	Leuk om te hebben maar niet nodig
22	Bij contact van het balletje met de omgeving dient er een geluidje te zijn	W	Leuk extra maar niet nodig voor de functionaliteit van het spel
23	Het spel houdt een highscore bij van vorige spelsessies	W	Leuk om te hebben maar niet nodig

4.0 Wireframes

Het spel kent in principe 3 schermen, het startscherm, het spelscherm en de eindscherm. Wij voegen hier nog een 4^{de} scherm aan toe om een level selecteerscherm te implementeren. Hieronder wordt toegelicht hoe elke scherm opgebouwd is en welke aspecten hierin voorkomen.

4.1 Startscherm

Het startscherm is het eerste scherm dat de speler te zien krijgt bij het openen van het spel. Vanuit hier kan de speler een nieuw spel starten, levels selecteren of het spel afsluiten.



Object	Beschrijving
1. Speltitel	De titel van het spel
2. Top score	Score die dynamisch wordt bijgehouden wanneer deze verbeterd wordt
3. Start spel knop	Om het spel te starten vanaf level 1
4. Level select knop	Om een ander level te selecteren om vanaf te starten
5. Stop spel knop	Om het spel af te sluiten

4.2 Level Selecteren

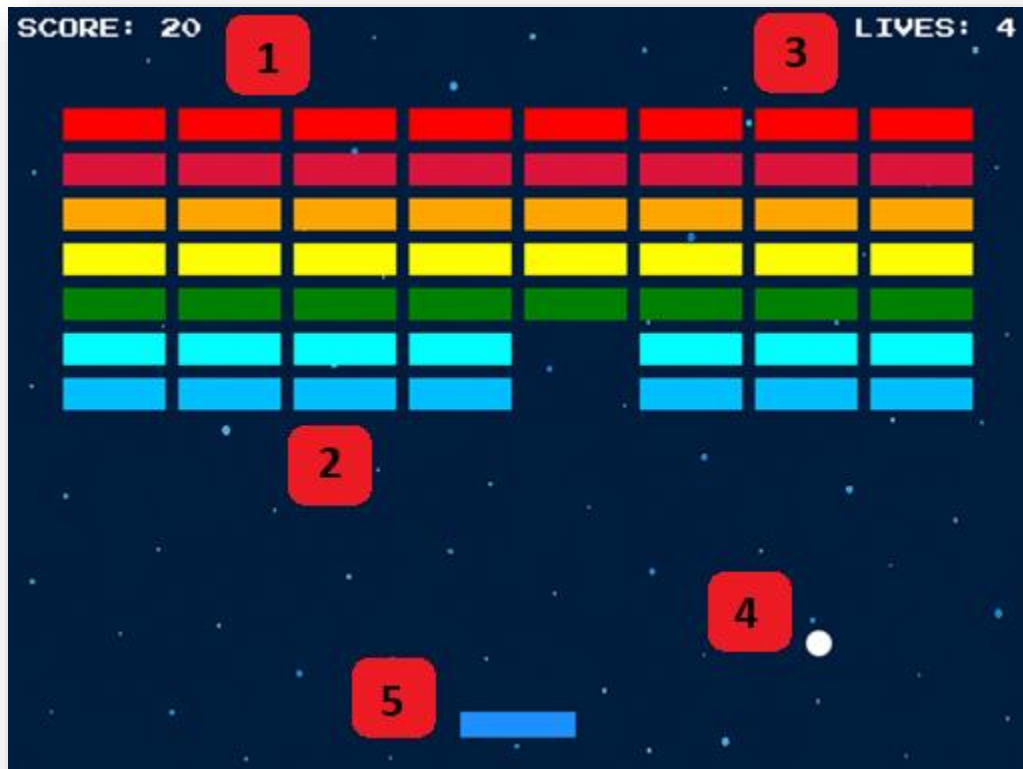
Het levelselectiescherm biedt de speler de mogelijkheid om direct een specifiek level te spelen. Hierdoor hoeft de speler niet telkens opnieuw vanaf level 1 te beginnen en kunnen moeilijke levels afzonderlijk geoefend worden.



Object	Beschrijving
1. Levels	Te selecteren levels, afhankelijk van de kleur wordt de moeilijkheidsgraad aangegeven. Hogere levels leveren meer punten op voor een highscore maar zijn lastiger te halen. Via dit scherm kan je moeilijke levels oefenen om later in een “run” het best mogelijke resultaat te behalen. Het selecteren van een level moet dan ook dat corresponderende level starten, wanneer dat level behaald is moet ook het volgende level het opvolgende level zijn en niet level 2

4.3 Spelscherm

Dit scherm vormt het belangrijkste onderdeel van het spel. Tijdens het spelen moet de speler reageren op de beweging van het balletje, strategisch blokken vernietigen en voorkomen dat het balletje onder het speelveld verdwijnt.



Object	Beschrijving
1. Score	De score wordt hier bijgehouden van het huidige spel. Verschillende acties leveren punten op, zoals blokken kapotmaken, levels halen en bonuspunten.
2. Blokken	De blokken zijn het doel van het spel, vernietig alle blokken en haal het level. Blokken hebben verschillende eigenschappen zoals de positie, hoe vaak een blok geraakt moet worden voordat deze kapot gaat. Het blok kan ook een power-up bevatten of onverwoestbaar zijn. De meeste van deze eigenschappen worden door de kleur gedefinieerd.
3. Levens	Hier worden de levens getoond, wanneer de levens 0 bereiken is het game-over

4. Balletje	Het balletje stuitert rond door het speelveld en heeft de kracht om de blokken te vernietigen, door middel van het platform wordt het balletje in het speelveld gehouden.
5. Platform "Vaus"	Het platform is door de speler te besturen in enkel horizontale richting en het balletje stuitert af van het platform, hiermee kan de richting van het balletje worden bepaald.

5.0 Technisch ontwerp

5.1 Inleiding

Het technisch ontwerp beschrijft hoe wij de aanpak hebben geprioriteerd en welke keuzes wij gemaakt hebben die wij realiseren in het spel op basis van de eerder gestelde requirements. Daarnaast is hier het klassendiagram te vinden die de basis vormt voor het spel evenals een uitleg over elke klasse die in het spel voorkomt.

5.2 Prioterisering

De MoSCoW methode vanuit het functioneel ontwerp wordt hier uitgevoerd en geprioteriseerd. Er wordt toegelicht welke punten die wel of niet in de scope staan van dit project.

Klassendiagram

Het klassendiagram bevat het gros van alle klassen in het spel, niet alle methoden en eigenschappen zijn erin opgenomen. Voorbeelden hiervan zijn interfaces en enkele superklassen die vanuit Yaeger gebruikt worden. Wij hebben gebruik gemaakt van PlantUML een schrijfwijze om zo klassendiagrammen te genereren, hierbij noteerden wij eerst alle klassen die wij verwachten nodig te gaan hebben en konden zo simpel relaties tekenen waar nodig en er zeker van zijn dat wij geen klassen vergeten.

Functionele eisen

De bedoeling is om het spel zo na te maken zoals het origineel gespeeld kon worden. Hieronder de eisen die wij minimaal van het spel eisen om zo dicht mogelijk bij het origineel te komen.

- Het spel moet afhankelijk van het level de juiste hoeveelheid blokken kunnen generen die wanneer ze in aanraking komen met het balletje een actie ondernemen
- Het spel moet te besturen zijn met de pijltjestoetsen van een toetsenbord
- Het spel moet een balletje intekenen die in rechte lijnen beweegt en bij botsing met een object in een gelijke hoek verder beweegt
- Het spel moet een platform intekenen die te besturen is door de speler
- Het spel moet een live score bijhouden
- Het spel moet bijhouden of het balletje het speelveld verlaat
- Het spel moet bijhouden of er balletjes op het speelveld bestaan
- Het spel moet wanneer er geen balletjes meer op het speelveld aanwezig zijn een game-over scherm tonen waarop de behaalde score te zien is
- De te verwoesten blokjes moeten willekeurig wel of niet een power-up activeren voor de speler
- Het spel moet willekeurig blokken genereren die een power-up bezitten of niet verwoestbaar zijn.
- Het spel moet afgekaderd zijn met onverwoestbare muren waar het balletje en het platform beiden niet doorheen kunnen

- Het spel moet een startscherm hebben
- Het balletje moet ook vanaf de zijkant van het platform kunnen stuiteren om zo de bal weer terug te sturen waar die vandaan kwam

Plan van aanpak

Om iteratief te werken hebben wij een plan opgesteld wat eerst gemaakt moest worden en welke klassen en functies eerst moesten bestaan voordat wij op een goede manier de rest van het spel kunnen implementeren. Ons plan is om eerst een startscherm en een spelscherm te maken, vanuit hier kunnen wij buttons, controls en blokjesgeneratie ontwikkelen en testen. Vanuit hier zijn wij op deze basis verder gaan werken.

1. Hoofdprogramma opzetten zodat er iets uit te voeren valt
2. Basis scenes implementeren, op dit moment nog leeg maar hier kan op gebouwd worden
3. Buttons aanmaken om tussen scenes te switchen en deze te implementeren
4. Blokjes genereren in de spelscene
5. Omringende muren opstellen en physics implementeren dat deze als boundaries van de spellevel dienen.
6. VAUS / padel opstellen en beweging toevoegen.
7. Balletje en physics van het balletje toevoegen, collision wordt vanuit het balletje berekend aangezien het balletje met elke entity in aanraking komt is het logisch om die logica bij het balletje te leggen.
8. Controller die de locatie van het balletje checkt of die nog in het speelveld is.
9. Houd levens bij, elke keer dat het balletje de scene verlaat -1 leven.
10. Alle physics laten werken, de padel moet de bal kunnen weerkaatsen en de bal moet de blokjes kapot kunnen maken.
11. Score implementeren per verwoest blokje
12. Game-Over scherm implementeren, een nieuw scene die na het behalen of verliezen van het level toont.
13. Nu dat het basis spelverloop is gerealiseerd kunnen de powerups geïmplementeerd worden.
14. Implementeer breder padel powerup.
15. Implementeer extra leven powerup.
16. Powerups kunnen willekeurig uit een vernietigd blokje komen.
17. Afhankelijk van de tijd kan er nog gekeken worden naar overige should-haves en would-haves om het spel nog uit te breiden.

5.3 Klassenbeschrijving

De klassenbeschrijving dient ter verduidelijking van het klassendiagram om correcte implementatie van de klassen te waarborgen. Door te beschrijven wat het doel en de functie is van een klasse is het duidelijk voor ontwikkelaars om de juiste besluiten te nemen voor het implementeren van de klasse, methode en functies die het omvatten.

Arkanoid

De klasse Arkanoid vormt het hoofdprogramma van het spel. Vanuit deze klasse worden alle scenes geregistreerd en kan er gewisseld worden tussen schermen zoals het hoofdmenu, het levelselectiescherm en het game-over scherm. Daarnaast wordt hier het spelvenster ingesteld en kan een nieuw spel gestart worden. Arkanoid beheert hiermee de algemene flow van het spel.

MainMenu

MainMenu is de eerste scene die zichtbaar wordt wanneer het spel gestart wordt. In deze scene worden de titel, highscore, knoppen en achtergrondmuziek weergegeven. Vanuit het hoofdmenu kan de speler het spel starten, een level selecteren of het spel afsluiten. Daarnaast wordt hier ook de topscore dynamisch bijgewerkt wanneer deze verbeterd wordt.

LevelSelector

De LevelSelector is de scene waarin de speler een level kan selecteren om te spelen. In deze scene worden de beschikbare levels weergegeven zodat de speler zelf een moeilijkheidsgraad kan kiezen. Daarnaast wordt hier achtergrondmuziek afgespeeld om dezelfde sfeer als het hoofdmenu te behouden. Deze scene vormt de verbinding tussen het hoofdmenu en de verschillende levels van het spel.

GameLevel

De GameLevel klasse bevat het daadwerkelijke spelverloop van Arkanoid. In deze scene worden alle belangrijke objecten zoals het platform, het balletje, de blokken en de gebruikersinterface aangemaakt. Daarnaast houdt deze klasse de score, levens en power-ups van de speler bij tijdens het spelen. Ook wordt hier gecontroleerd of de speler gewonnen heeft of game-over is gegaan waarna het juiste scherm wordt geopend.

GameOverScene

De GameOverScene wordt weergegeven wanneer de speler het spel heeft verloren of een level succesvol heeft afgerond. In deze scene worden de behaalde score en de hoogste score van het spel getoond. Daarnaast krijgt de speler de mogelijkheid om terug te keren naar het hoofdmenu. Deze scene vormt daarmee de afsluiting van een speelsessie.

Button

De Button klasse vormt de basis voor alle knoppen binnen het spel. Deze klasse bevat de algemene functionaliteit zoals tekstweergave, kleuren en hover-effecten. Wanneer de speler met de muis over een knop beweegt verandert de kleur en cursor automatisch. Andere knoppen binnen het spel erven deze functionaliteit zodat dezelfde stijl overal gebruikt kan worden.

QuitButton

De QuitButton is een subklasse van Button en wordt gebruikt om het spel af te sluiten. Wanneer de speler op deze knop drukt wordt de applicatie volledig beëindigd.

StartButton

De StartButton is een subklasse van Button en wordt gebruikt om een nieuw spel te starten. Wanneer de speler op deze knop drukt wordt het eerste level geladen.

BackToMenuButton

De BackToMenuButton is een subklasse van Button en wordt gebruikt om terug te keren naar het hoofdmenu. Hiermee kan de speler vanuit andere schermen weer terug navigeren.

SelectLevelButton

De SelectLevelButton is een subklasse van Button en wordt gebruikt om naar het levelselectiescherm te gaan. Vanuit hier kan de speler een level kiezen om te spelen.

Paddle

De Paddle klasse stelt het platform van de speler voor binnen het spel. Het platform kan horizontaal bewogen worden met de pijltjestoetsen en zorgt ervoor dat het balletje terug het speelveld in wordt gekeerd. Daarnaast controleert deze klasse of het platform binnen de grenzen van het speelveld blijft. Ook kunnen power-ups tijdelijk de grootte van het platform aanpassen.

Ball

De Ball klasse bestuurt het balletje binnen het spel en verwerkt alle botsingen met objecten. Het balletje stuitert tegen muren, blokken en het platform om zo door het speelveld te bewegen. Daarnaast controleert deze klasse wanneer een blok geraakt of vernietigd wordt en wanneer de speler een leven verliest. Tijdens het spel kan de snelheid van het balletje toenemen om het spel uitdagender te maken.

Brick

Object wat kapot kan wanneer die in aanraking komt met het balletje, een brick heeft een willekeurig aantal aanrakingen nodig om kapot te gaan. Wanneer de brick kapot gaat is er een kans op een powerup te generen.

Powerup

De Powerup klasse stelt tijdelijke extra effecten voor die de speler kan verzamelen tijdens het spel. Power-ups verschijnen willekeurig wanneer een blok vernietigd wordt en vallen vervolgens naar beneden richting het platform. Wanneer de speler een power-up opvangt wordt het bijbehorende effect direct geactiveerd. Verschillende power-ups kunnen bijvoorbeeld extra levens of een groter platform geven.

PowerupConfig

De PowerupConfig klasse bevat de instellingen van verschillende power-ups binnen het spel. Vanuit deze klasse worden waarden zoals de kans op een power-up, de duur van effecten en de grootte van het platform beheerd. Hierdoor kunnen gameplay-instellingen eenvoudig aangepast worden zonder de overige code te veranderen. Deze klasse zorgt daarmee voor een overzichtelijke configuratie van de power-up functionaliteit.

PowerupType

De PowerupType klasse bevat de verschillende soorten power-ups die in het spel beschikbaar zijn. Elke power-up heeft een eigen effect en een eigen kleur zodat deze herkenbaar is voor de speler. Vanuit deze klasse wordt bepaald wat er gebeurt wanneer een speler een power-up opvangt. Hierdoor kunnen eenvoudig nieuwe power-ups aan het spel toegevoegd worden.

PowerupIndicator

De PowerupIndicator klasse toont een korte visuele melding wanneer een power-up geactiveerd wordt. Hierbij verschijnt tijdelijk een symbool op de locatie waar het blok vernietigd is. De kleur en het symbool verschillen per type power-up zodat de speler direct kan zien welke bonus actief is geworden. Dit zorgt voor extra feedback tijdens het spelen.

GameText

De GameText klasse vormt de basis voor alle teksten die tijdens het spel weergegeven worden. Deze klasse bevat de algemene stijl van de gebruikersinterface zoals lettertype en kleur. Andere tekstklassen binnen het spel maken gebruik van deze basis zodat alle informatie op een consistente manier wordt weergegeven.

ScoreText

De ScoreText klasse is een subklasse van GameText en toont de huidige score van de speler tijdens het spel. De score wordt dynamisch bijgewerkt wanneer de speler punten verdient door bijvoorbeeld blokken te vernietigen. Deze tekst wordt weergegeven in de gebruikersinterface van het spel zodat de speler altijd de actuele score kan zien.

LivesText

De LivesText klasse is een subklasse van GameText en toont hoeveel levens de speler nog over heeft tijdens het spel. Wanneer het balletje onder het speelveld verdwijnt wordt het aantal levens aangepast. Deze informatie wordt zichtbaar weergegeven in de gebruikersinterface zodat de speler altijd kan zien hoeveel kansen er nog over zijn.

FileManager

De FileManager klasse verzorgt het opslaan en uitlezen van gegevens binnen het spel. Deze klasse wordt gebruikt om informatie zoals de highscore tussen verschillende spelsessies te bewaren. Door gebruik te maken van een apart bestand blijven gegevens behouden wanneer het spel opnieuw gestart wordt. Hierdoor kan de speler zijn behaalde topscore terugzien in het hoofdmenu.

FontManager

De FontManager klasse beheert de lettertypes die binnen het spel gebruikt worden. Vanuit deze klasse wordt het juiste lettertype geladen zodat teksten overal dezelfde stijl behouden. Hierdoor krijgt het spel een consistente retro uitstraling die past bij het originele Arkanoid.

6.0 Conclusie

Het maken van het spel beviel ons goed en gaf een goed beeld van hoe object georiënteerd programmeren toegepast kan worden binnen game development. De ondersteuning van Yaeger was grotendeels prettig om mee te werken, al liepen wij tegen enkele uitdagingen aan bij het implementeren van de collision van het balletje tegen de blokken en het platform. Hierdoor ontstond soms onverwacht gedrag tijdens het spelen.

Wij hebben alle functionele eisen die onder de Must-Haves vielen succesvol geïmplementeerd en daarnaast ook een aantal Should-Haves gerealiseerd. Ook is de huidige structuur van het spel geschikt gemaakt voor toekomstige uitbreidingen zoals extra power-ups en nieuwe levels. Hierdoor kan het spel relatief eenvoudig verder uitgebreid worden zonder grote aanpassingen aan de bestaande code.

Daarnaast zijn wij begonnen met het ontwikkelen van een levelselectiescherm, maar deze functionaliteit was net niet volledig afgerond voor de deadline. Hierdoor zijn de extra levels uiteindelijk nog niet toegevoegd aan het spel. Met meer tijd hadden deze uitbreidingen extra variatie en diepgang aan het spelverloop kunnen geven. Ondanks deze punten zijn wij tevreden met het eindresultaat en de functionaliteiten die gerealiseerd zijn.

7.0 Bijlage

7.1 Klassendiagram

Hieronder is het klassendiagram van het spel weergegeven. In dit diagram zijn de belangrijkste klassen, eigenschappen en relaties binnen het project opgenomen. Het klassendiagram geeft een overzicht van de structuur van het spel en laat zien hoe de verschillende onderdelen met elkaar samenwerken.

